G.H. Raisoni College of Engineering and Management
Department of Mechanical Engineering

Unit 4 :Linked List

Prof. Sagar Mahajan

DYNAMIC MEMORY ALLOCATION

TOPIC 1: Dynamic Memory Allocation

Analogy



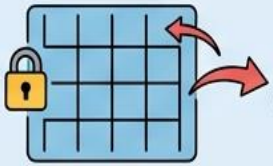
Fixed Almirah (Array)
Size fixed in advance.
5 or 50 tools? Don't know!



Adding Shelves as needed
Add space as you need it.
That's dynamic memory allocation!

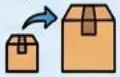
Concept

Static (array)

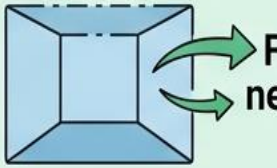


Size fixed in advance

- If you need more, you must create a bigger array and copy everything.



Dynamic



- Request memory at runtime when needed.
- When done, release it.

Request it
Release it

4 Functions in C

Function	Purpose	Example
malloc	Mallot the demoantable memory allocation	**malloc**(*nteran); **malloc**(size, [ɣernent));
calloc	Callocit memory at space when done ɓanmor create it	**calloc**(c new[* *0); **calloc**(c nz=10);
realloc	Relioie memory at attacs kndoft nates to create	**realloc**(c new[* *i], chan);
free	Pesase memory at runtime reliase release.	**free**();

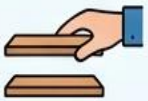
Code (line-by-line)

```
1  #include <stra.h>
2  {
3      malloc = malloc(fixed, Sized=120);
4      calloc = calloc(nated, calloc)
5      realloc = realloc(xed, *realloc_[aatec], as "someonee it", %0%8d);
6      free("free", null);
7      return 0;
8  }
```

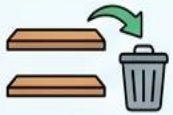
Key Points for Mechanical Students



Array = fixed-size rack



malloc = "I need a new shelf"



free = "Clear the shelf, someone else needs it"



This concept will be used in linked lists

DYNAMIC MEMORY ALLOCATION

TOPIC 1: Dynamic Memory Allocation

Analogy



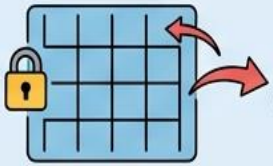
Fixed Almirah (Array)
Size fixed in advance.
5 or 50 tools? Don't know!



Adding Shelves as needed
Add space as you need it.
That's dynamic memory allocation!

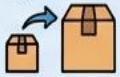
Concept

Static (array)

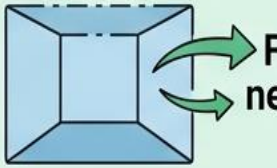


Size fixed in advance

- If you need more, you must create a bigger array and copy everything.



Dynamic



- Request memory at runtime when needed.
- When done, release it.

Request it
Release it

4 Functions in C

Function	Purpose	Example
malloc	Mallot the demoantable memory allocation	**malloc**(*nteran); **malloc**(size, [ɣernent));
calloc	Callocit memory at space when done ɓanmor create it	**calloc**(c new[* *0); **calloc**(c nz=10);
realloc	Relioie memory at attacs kndoft nates to create	**realloc**(c new[* *i], chan);
free	Pesase memory at runtime reliase release.	**free**();

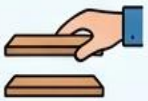
Code (line-by-line)

```
1  #include <stra.h>
2  {
3      malloc = malloc(fixed, Sized=120);
4      calloc = calloc(nated, calloc)
5      realloc = realloc(xed, *realloc_[aatec], as "someonee it", %0%8d);
6      free("free", null);
7      return 0;
8  }
```

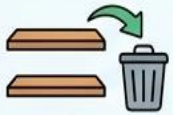
Key Points for Mechanical Students



Array = fixed-size rack



malloc = "I need a new shelf"



free = "Clear the shelf, someone else needs it"

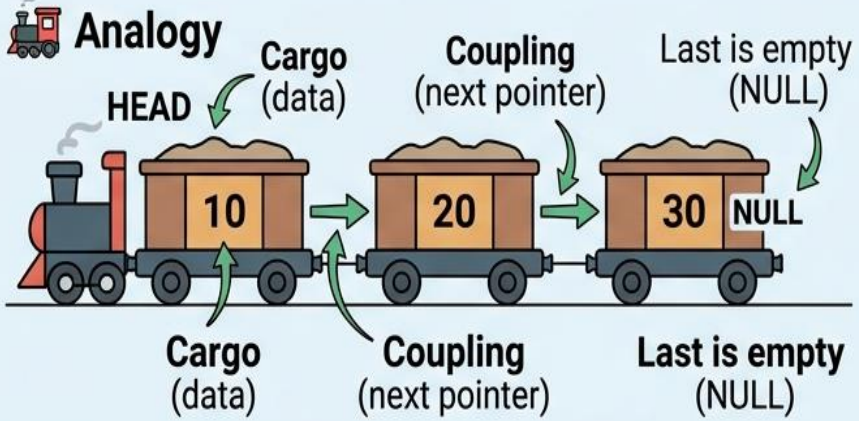


This concept will be used in linked lists

SINGLY LINKED LIST

TOPIC 2: Singly Linked List

Analogy



Concept

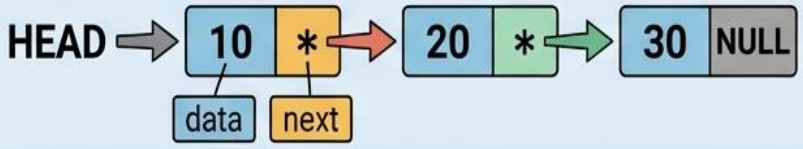
In an Array
(contiguous)



In a Linked List
(non-contiguous)



Diagram



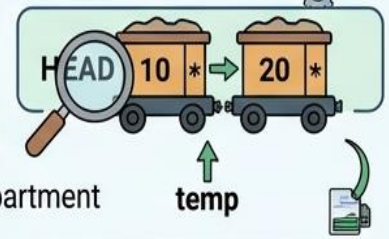
Node Structure

```
1 struct Node {
2   int data;          // value
3   struct Node* next; // address
4 };
```

`struct Node` – creates a new compartment
`int data` – space for a number
`struct Node* next` – space for address of next compartment



Traversal




```
1 void traverse(it temp) {
2   return temp);
```

10 -> 20 -> 30 -> NULL

Insertion & Deletion (Cases)

Insertion (3 Cases)

Case A: (Beginning)
Before: HEAD -> [20] -> [30]
After: HEAD -> [10] -> [20] -> [30]

Case B: (End)

Case C: (Position)

Deletion (3 Cases)

Case A: (Beginning)

Case B: (End) → **free**(temp)

Case C: (Position): **free**(temp)

Remember



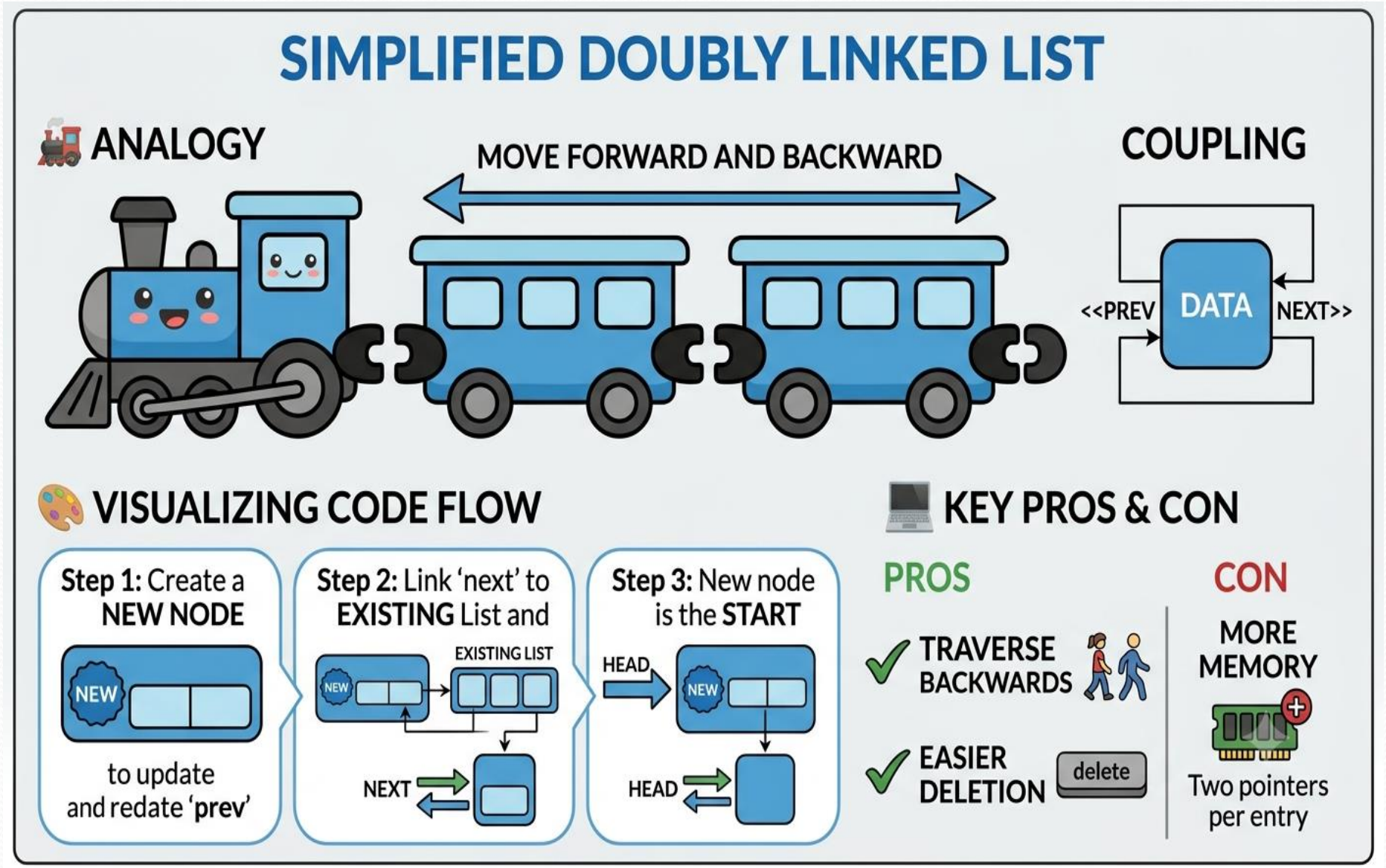
Access
O(n) - traverse from start

Insert at start
O(1)

Insert at end
O(n) - must go to the end)

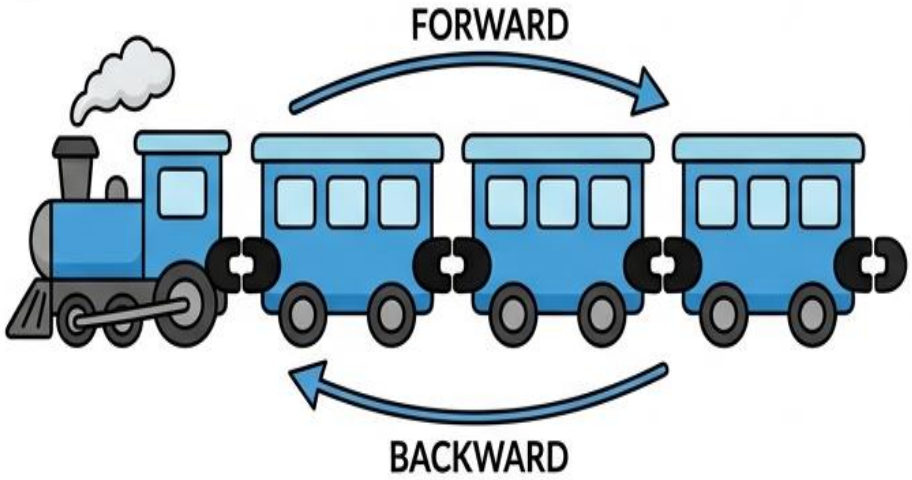
Delete: Same complexity (O(n) at end, O(1) at start)

DOUBLY LINKED LIST



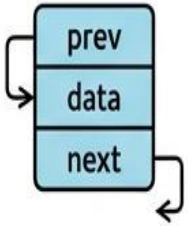
DOUBLY LINKED LIST EXPLAINED

ANALOGY



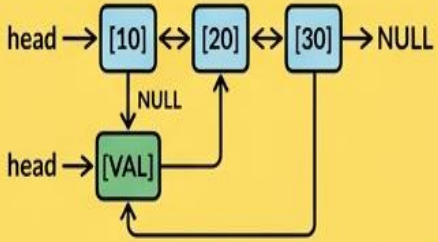
NODE (Struct)

```
struct Node {
   int data;
   struct Node* prev;  // pointer to previous node
   struct Node* next;  // pointer to next node
};
```

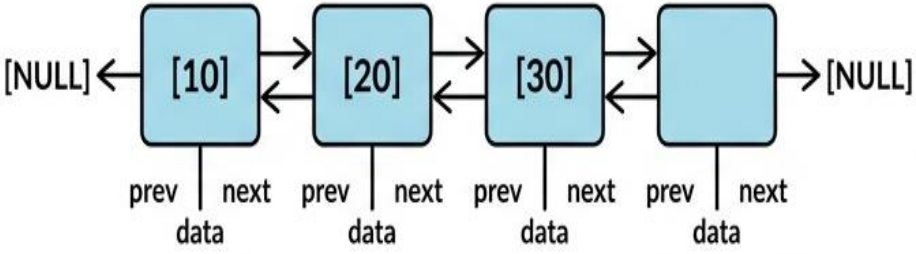


INSERT AT BEGINNING

```
struct Node* insertBegin(struct Node* head, int val) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = val;
   newNode->prev = NULL;
   newNode->next = head;
   if (head != NULL)
      head->prev = newNode;
   return newNode;
}
```



DIAGRAM



ADVANTAGES & DISADVANTAGES

ADVANTAGES

- ✓ Backward traversal possible
- ✓ Deletion easier (we have prev pointer)

DISADVANTAGES

- ✗ Extra memory (2 pointers per node)

CIRCULAR LINKED LIST

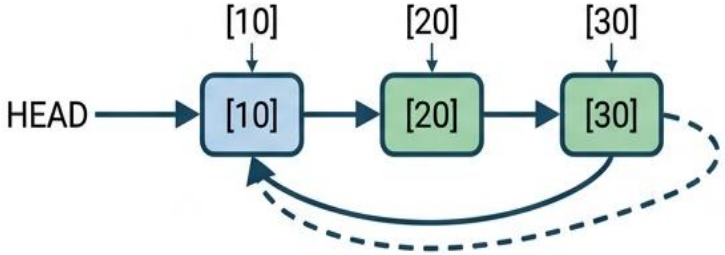
TOPIC 4: Circular Linked List

🚂 **Analogy:** Round-robin ride.



The last compartment connects back to the first compartment. **No NULL. No end.**

🎨 **Diagram:**



⚠️ **Common Mistake:**

`while (temp != NULL)` is wrong — NULL never appears, infinite loop

💻 **Node** (same as singly):

```
struct Node {
    int data;
    struct Node* next;
};
```

💻 **Traversal** (special case!):

```
void traverse(struct Node* head) {
  if (head == NULL) return;
  struct Node* temp = head;
  do {
     printf("%d -> ", temp->data);
     temp = temp->next;
  } while (temp != head);  // stop when we return to head
  printf("(back to head)\n");
}
```

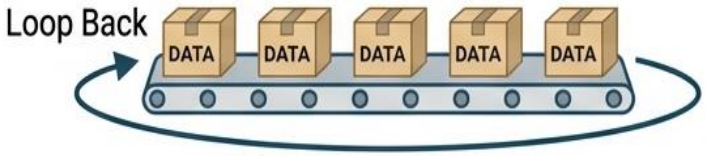
🔑 **Use Cases:**

- Round-robin CPU scheduling
- Music player loop
- Multiplayer board games (turn cycle)

CIRCULAR LINKED LIST (CREATION)

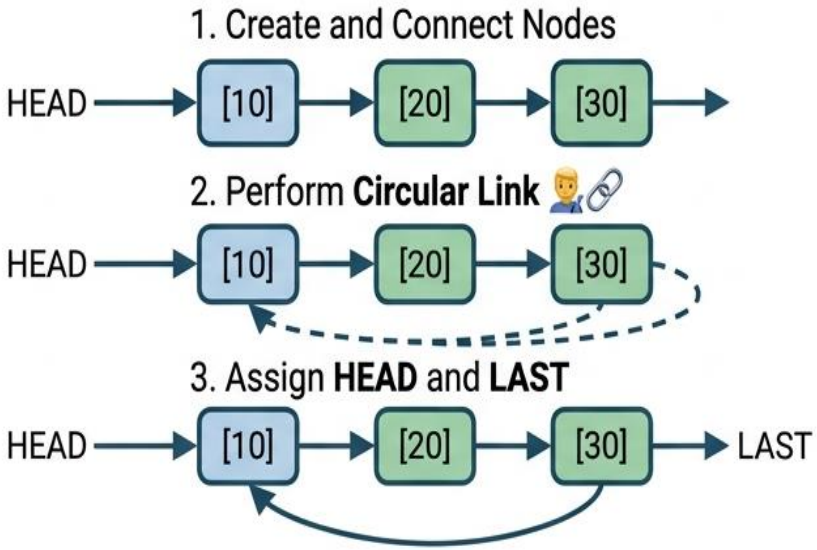
TOPIC 4.1: Creation of Circular Linked List

🚗 Process Analogy: A Closed Conveyor Belt



The conveyor forms a complete loop.
New items are inserted into the cycle.

🎬 Step-by-Step Creation Diagram:



💻 Implementation Code: C Language

```c
struct Node* createCircularList(int data[], int n) {
    if (n == 0) return NULL;

    struct Node *head = NULL, *temp = NULL, *newNode;

    for (int i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = data[i];
        if (head == NULL) {
            head = newNode;
        } else {
            temp->next = newNode;
        }
        temp = newNode;
    }
    // THE CIRCULAR LINK step
    if (temp != NULL) {
        temp->next = head; // Make last node point to head
    }
    return head;
}
```

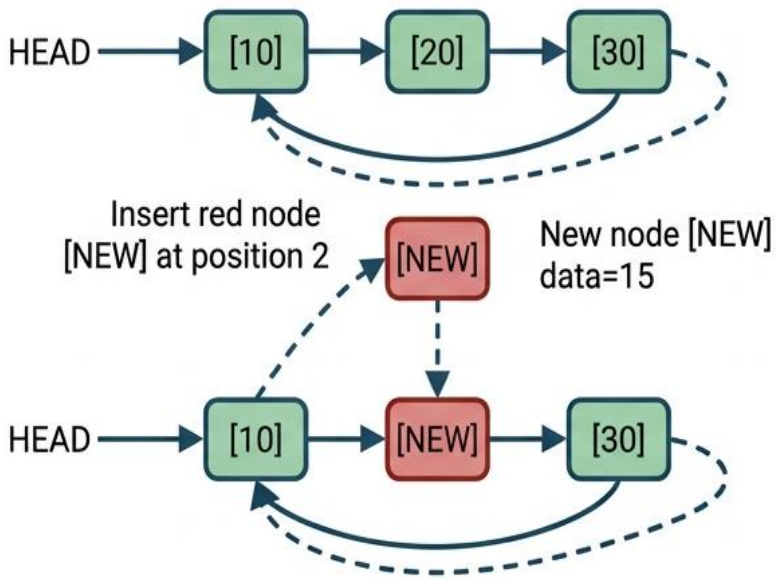
⚠️ Common Pitfalls:

- Forgetting to link LAST back to HEAD: Results in a standard, finite singly linked list, not a cycle.
- Not checking for empty list (n==0): Can cause memory allocation errors or dereferencing NULL.

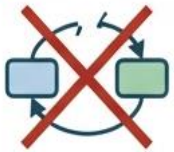
CIRCULAR LINKED LIST (INSERTION)

INSERTION IN CIRCULAR LINKED LIST

🎨 Diagram:



⚠️ Common Mistake:



Inserting a node without connecting the last element back to HEAD.

💻 C Code Snippet

```
struct Node *insertPosition(struct Node *head, int data
    struct Node *new_node, *current;
    int i;
    if (head == NULL) { // Handle empty list
        // special case
    }
    new_node = malloc(sizeof(struct Node));
    new_node->data = data;
    current = head;
    for (i = 0; i < pos - 1; i++) {
        current = current->next;
        if (current == head) break; // End of list
    }
    new_node->next = current->next;
    current->next = new_node;
    return head;
}
```

🔑 Complexity

- Time: O(n) for searching position, O(1) for insertion;
- Space: O(1) for new node.

ARRAY V/S LINKED LIST

